

CS122 Final Group Projects: The Brainstorm and Example Guide

1. BUSINESS/CORPORATE APPLICATION

Project Ideas:

- **Inventory Management System:** Track products, stock levels, reorder alerts
- **Employee Schedule Planner:** Create/manage shift schedules, prevent conflicts
- **Service Booking System:** Handle appointments, prevent double-booking
- **Task/Project Management Dashboard:** Assign tasks, track progress, set deadlines

Real-World Value:

Small businesses often can't afford expensive software—a custom tool saves money and improves efficiency. Scalable for SaaS monetization.

Java Example:

```
// Core OOP concept: Encapsulation + Collections
public class Inventory {
    private ArrayList<Product> products;
    private String businessName;

    public Inventory(String businessName) {
        this.businessName = businessName;
        this.products = new ArrayList<>();
    }

    public void addProduct(Product p) {
        products.add(p);
    }

    public void updateStock(String productId, int quantity) {
        for (Product p : products) {
            if (p.getId().equals(productId)) {
                p.setStock(p.getStock() + quantity);
                if (p.getStock() < p.getMinThreshold()) {
                    sendReorderAlert(p);
                }
            }
        }
    }

    public ArrayList<Product> getLowStockItems() {
        ArrayList<Product> lowStock = new ArrayList<>();
        for (Product p : products) {
            if (p.getStock() < p.getMinThreshold()) {
```

```
        lowStock.add(p);
    }
}
return lowStock;
}

private void sendReorderAlert(Product p) {
    System.out.println("⚠ ALERT: " + p.getName() + " is low on
stock!");
}
}

public class Product {
    private String id;
    private String name;
    private int stock;
    private int minThreshold;
    private double price;

    public Product(String id, String name, int stock, int minThreshold,
double price) {
        this.id = id;
        this.name = name;
        this.stock = stock;
        this.minThreshold = minThreshold;
        this.price = price;
    }

    // Getters and setters
    public String getId() { return id; }
    public String getName() { return name; }
    public int getStock() { return stock; }
    public void setStock(int stock) { this.stock = stock; }
    public int getMinThreshold() { return minThreshold; }
    public double getPrice() { return price; }
}
```

Why This Works: Demonstrates ArrayList, encapsulation, and method logic we've learned throughout the semester.

Example Demo Usage & Output:

```
public class InventoryDemo {
    public static void main(String[] args) {
        Inventory shop = new Inventory("Bob's Coffee Shop");

        // Add products
        shop.addProduct(new Product("P001", "Coffee Beans", 25, 5,
12.99));
        shop.addProduct(new Product("P002", "Paper Cups", 100, 20, 0.50));
        shop.addProduct(new Product("P003", "Milk", 8, 10, 3.99));
    }
}
```

```
System.out.println("=== Initial Stock ===");
System.out.println("Added 3 products to inventory\n");

// Update stock (selling items)
System.out.println("=== Selling Items ===");
shop.updateStock("P002", -85); // Sold 85 cups
shop.updateStock("P001", -18); // Sold 18 bags of coffee
shop.updateStock("P003", -5); // Sold 5 cartons of milk

// Check low stock
System.out.println("\n=== Low Stock Items ===");
ArrayList<Product> lowItems = shop.getLowStockItems();
for (Product p : lowItems) {
    System.out.println("- " + p.getName() + ": " + p.getStock() +
" units");
}
}
```

Output:

```
=== Initial Stock ===
Added 3 products to inventory

=== Selling Items ===
⚠ ALERT: Paper Cups is low on stock!
⚠ ALERT: Milk is low on stock!

=== Low Stock Items ===
- Paper Cups: 15 units
- Milk: 3 units
```

2. SECURITY APPLICATION

Project Ideas:

- **Password Manager with Basic Encryption:** Store and encrypt user credentials
- **Secure Note-Taking App:** Encrypt notes, password-protected access
- **File Encryption Tool:** Simple encryption/decryption for sensitive documents
- **User Authentication System:** Validate login with encrypted passwords

Real-World Value:

Everyone needs password management. Cybersecurity tools are high-demand and can be monetized through subscriptions.

Java Example:

```
// Core OOP: Inheritance + Encapsulation + Java encryption basics
import java.security.MessageDigest;
import java.util.HashMap;

public class PasswordManager {
    private HashMap<String, StoredCredential> credentials; // HashMap for
0(1) lookups
    private String masterPassword;

    public PasswordManager(String masterPassword) {
        this.masterPassword = masterPassword;
        this.credentials = new HashMap<>();
    }

    public void storeCredential(String service, String username, String
password) {
        if (validateMasterPassword()) {
            String hashedPassword = hashPassword(password);
            credentials.put(service, new StoredCredential(username,
hashedPassword));
            System.out.println("✓ Credential stored for " + service);
        } else {
            System.out.println("✗ Invalid master password");
        }
    }

    public StoredCredential retrieveCredential(String service) {
        return credentials.get(service);
    }

    private String hashPassword(String password) {
        try {
            MessageDigest md = MessageDigest.getInstance("SHA-256");
            byte[] hash = md.digest(password.getBytes());
            StringBuilder hexString = new StringBuilder();
            for (byte b : hash) {
                String hex = Integer.toHexString(0xff & b);
                if (hex.length() == 1) hexString.append('0');
                hexString.append(hex);
            }
            return hexString.toString();
        } catch (Exception e) {
            e.printStackTrace();
        }
        return null;
    }

    private boolean validateMasterPassword() {
        // Simple validation – in real app, prompt user input
        return true;
    }
}
```

```

public class StoredCredential {
    private String username;
    private String encryptedPassword;

    public StoredCredential(String username, String encryptedPassword) {
        this.username = username;
        this.encryptedPassword = encryptedPassword;
    }

    public String getUsername() { return username; }
    public String getEncryptedPassword() { return encryptedPassword; }
}

```

Why This Works: Uses HashMap, demonstrates hashing concept, shows practical security application.

Example Demo Usage & Output:

```

public class PasswordManagerDemo {
    public static void main(String[] args) {
        PasswordManager manager = new PasswordManager("MyMasterPass123");

        System.out.println("=== Storing Credentials ===");
        manager.storeCredential("Gmail", "sarah@example.com",
"SecurePass456");
        manager.storeCredential("GitHub", "sarah_dev", "GitHubKey789");
        manager.storeCredential("Bank", "sarah.brown", "BankPass000");

        System.out.println("\n=== Retrieving Stored Credentials ===");
        StoredCredential gmail = manager.retrieveCredential("Gmail");
        if (gmail != null) {
            System.out.println("Service: Gmail");
            System.out.println("Username: " + gmail.getUsername());
            System.out.println("Password (hashed): " +
gmail.getEncryptedPassword().substring(0, 20) + "...");
        }

        System.out.println("\nService: GitHub");
        StoredCredential github = manager.retrieveCredential("GitHub");
        System.out.println("Username: " + github.getUsername());

        System.out.println("\n✓ All credentials encrypted and stored
safely");
    }
}

```

Output:

```
=== Storing Credentials ===
✓ Credential stored for Gmail
✓ Credential stored for GitHub
✓ Credential stored for Bank

=== Retrieving Stored Credentials ===
Service: Gmail
Username: sarah@example.com
Password (hashed): e3b0c44298fc1c14a...

Service: GitHub
Username: sarah_dev

✓ All credentials encrypted and stored safely
```

3. GAME DEVELOPMENT

Project Ideas:

- **Text-Based Adventure Game:** Choose-your-own-adventure with inventory system
- **Simple 2D Game:** Snake, Breakout, or Space Invaders (using Swing/JFrame)
- **Card Game:** Poker, Blackjack, or Solitaire with dealer logic
- **Dungeon Crawler:** Text-based RPG with player stats, combat, leveling

Real-World Value:

Games are an established market. Prototypes can be monetized through app stores or ad revenue.

Java Example:

```
// Core OOP: Inheritance + Polymorphism + Game State Management
public abstract class GameCharacter {
    protected String name;
    protected int health;
    protected int attackPower;

    public GameCharacter(String name, int health, int attackPower) {
        this.name = name;
        this.health = health;
        this.attackPower = attackPower;
    }

    public abstract void attack(GameCharacter opponent);

    public void takeDamage(int damage) {
        this.health -= damage;
        System.out.println(name + " takes " + damage + " damage! Health: "
+ health);
    }
}
```

```
        public boolean isAlive() {
            return health > 0;
        }
    }

    public class Player extends GameCharacter {
        private int level;
        private ArrayList<Item> inventory;

        public Player(String name, int health, int attackPower) {
            super(name, health, attackPower);
            this.level = 1;
            this.inventory = new ArrayList<>();
        }

        @Override
        public void attack(GameCharacter opponent) {
            int damage = attackPower + (level * 2); // Scaling with level
            System.out.println(name + " attacks " + opponent.name + " for " +
damage + " damage!");
            opponent.takeDamage(damage);
        }

        public void levelUp() {
            level++;
            health += 20;
            attackPower += 5;
            System.out.println("📈 " + name + " leveled up to Level " +
level);
        }

        public void addItem(Item item) {
            inventory.add(item);
        }
    }

    public class Enemy extends GameCharacter {
        public Enemy(String name, int health, int attackPower) {
            super(name, health, attackPower);
        }

        @Override
        public void attack(GameCharacter opponent) {
            System.out.println(name + " attacks " + opponent.name + " for " +
attackPower + " damage!");
            opponent.takeDamage(attackPower);
        }
    }

    public class Item {
        private String name;
        private int value;
```

```
public Item(String name, int value) {
    this.name = name;
    this.value = value;
}

public String getName() { return name; }
public int getValue() { return value; }
}
```

Why This Works: Demonstrates inheritance, polymorphism, and ArrayList usage.

Example Demo Usage & Output:

```
public class GameDemo {
    public static void main(String[] args) {
        Player hero = new Player("Aragorn", 100, 15);
        Enemy goblin = new Enemy("Goblin", 30, 5);

        System.out.println("=== DUNGEON ADVENTURE ===\n");
        System.out.println("Hero: " + hero.name + " (HP: " + hero.health +
        ")");
        System.out.println("Enemy: " + goblin.name + " (HP: " +
        goblin.health + ")\n");

        // Battle simulation
        System.out.println("=== BATTLE BEGINS ===");
        hero.attack(goblin);
        goblin.attack(hero);

        hero.attack(goblin);
        goblin.attack(hero);

        hero.attack(goblin);

        if (!goblin.isAlive()) {
            System.out.println("\n" + goblin.name + " is defeated!");
            hero.levelUp();
            hero.addItem(new Item("Gold Coin", 10));
            System.out.println("Hero gained: Gold Coin");
        }

        System.out.println("\nHero final stats: HP = " + hero.health + ",
        Level = " + hero.level);
    }
}
```

Output:


```
=== DUNGEON ADVENTURE ===

Hero: Aragorn (HP: 100)
Enemy: Goblin (HP: 30)

=== BATTLE BEGINS ===
Aragorn attacks Goblin for 20 damage!
Goblin takes 20 damage! Health: 10
Goblin attacks Aragorn for 5 damage!
Aragorn takes 5 damage! Health: 95
Aragorn attacks Goblin for 20 damage!
Goblin takes 20 damage! Health: -10

Goblin is defeated!
👤 Aragorn leveled up to Level 2
Hero gained: Gold Coin

Hero final stats: HP = 95, Level = 2
```

4. HEALTH/HEALTHCARE APPLICATION

Project Ideas:

- **Patient Queue Management:** Track check-in order, estimated wait times
- **Appointment Scheduler:** Book, reschedule, send reminders
- **Health Tracker:** Log meals, exercise, water intake with analytics
- **Medication Reminder System:** Track medications with dosages and schedules

Real-World Value:

Healthcare is regulated but in-demand. Even simple tools reduce administrative overhead for clinics and help patients track health.

Java Example:

```
// Core OOP: Encapsulation + Collections + File I/O
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;

public class PatientQueue {
    private ArrayList<Patient> queue;
    private int estimatedWaitTime; // in minutes

    public PatientQueue(int estimatedWaitTime) {
        this.queue = new ArrayList<>();
        this.estimatedWaitTime = estimatedWaitTime;
    }
}
```

```

        public void checkIn(Patient patient) {
            patient.setCheckInTime(LocalDateTime.now());
            queue.add(patient);
            System.out.println("✓ " + patient.getName() + " checked in.
Estimated wait: " +
                                (queue.size() * estimatedWaitTime) + "
minutes");
        }

        public Patient callNextPatient() {
            if (queue.isEmpty()) {
                System.out.println("No patients in queue");
                return null;
            }
            Patient next = queue.remove(0);
            next.setSeenTime(LocalDateTime.now());
            System.out.println("→ " + next.getName() + " called to room");
            return next;
        }

        public ArrayList<Patient> getWaitingPatients() {
            return queue;
        }

        public int getQueueLength() {
            return queue.size();
        }
    }

    public class Patient {
        private String id;
        private String name;
        private String condition;
        private LocalDateTime checkInTime;
        private LocalDateTime seenTime;

        public Patient(String id, String name, String condition) {
            this.id = id;
            this.name = name;
            this.condition = condition;
        }

        public String getName() { return name; }
        public String getCondition() { return condition; }
        public void setCheckInTime(LocalDateTime time) { this.checkInTime =
time; }
        public void setSeenTime(LocalDateTime time) { this.seenTime = time; }
        public LocalDateTime getCheckInTime() { return checkInTime; }
        public long getWaitTimeMinutes() {
            if (checkInTime == null || seenTime == null) return 0;
            return java.time.temporal.ChronoUnit.MINUTES.between(checkInTime,
seenTime);
        }
    }
}

```

```
public class Appointment {
    private String patientName;
    private LocalDateTime appointmentTime;
    private String doctorName;
    private String status; // "scheduled", "completed", "cancelled"

    public Appointment(String patientName, LocalDateTime time, String
doctorName) {
        this.patientName = patientName;
        this.appointmentTime = time;
        this.doctorName = doctorName;
        this.status = "scheduled";
    }

    public boolean isUpcoming() {
        return appointmentTime.isAfter(LocalDateTime.now());
    }

    public void reschedule(LocalDateTime newTime) {
        this.appointmentTime = newTime;
        System.out.println("✓ Appointment rescheduled to " + newTime);
    }
}
```

Why This Works: Shows practical healthcare application, uses ArrayList for patient management.

Example Demo Usage & Output:

```
public class HealthcareDemo {
    public static void main(String[] args) {
        PatientQueue clinic = new PatientQueue(15); // 15 min per patient

        System.out.println("=== CLINIC PATIENT QUEUE ===\n");

        // Check-in patients
        clinic.checkIn(new Patient("P001", "John Smith", "Flu symptoms"));
        clinic.checkIn(new Patient("P002", "Maria Garcia", "Back pain"));
        clinic.checkIn(new Patient("P003", "Ahmed Hassan", "Checkup"));
        clinic.checkIn(new Patient("P004", "Lisa Chen", "Headache"));

        System.out.println("\nQueue length: " + clinic.getQueueLength() +
" patients\n");

        // Call patients to see doctor
        System.out.println("=== CALLING PATIENTS ===");
        Patient current = clinic.callNextPatient();
        System.out.println("Waiting time for " + current.getName() + ": "
+ current.getWaitTimeMinutes() + " minutes");

        clinic.callNextPatient();
    }
}
```

```
        clinic.callNextPatient();

        // Show remaining
        System.out.println("\n=== REMAINING PATIENTS ===");
        ArrayList<Patient> waiting = clinic.getWaitingPatients();
        for (Patient p : waiting) {
            System.out.println("- " + p.getName() + " (" +
                p.getCondition() + ")");
        }
    }
}
```

Output:

```
=== CLINIC PATIENT QUEUE ===

✓ John Smith checked in. Estimated wait: 15 minutes
✓ Maria Garcia checked in. Estimated wait: 30 minutes
✓ Ahmed Hassan checked in. Estimated wait: 45 minutes
✓ Lisa Chen checked in. Estimated wait: 60 minutes

Queue length: 4 patients

=== CALLING PATIENTS ===
→ John Smith called to room
Waiting time for John Smith: 2 minutes

→ Maria Garcia called to room
→ Ahmed Hassan called to room

=== REMAINING PATIENTS ===
- Lisa Chen (Headache)
```

5. ENTERTAINMENT/SOCIAL MEDIA/STREAMING/CREATOR CULTURE & SPORTS APPLICATION

Project Ideas:

Content Creator/Streaming Focus:

- **Content Management Platform:** Organize videos, track engagement metrics, manage playlists
- **Livestream Scheduler:** Schedule streams, notify followers, track viewer count
- **Podcast/Music Library:** Catalog episodes/songs, manage playlists, track listening history
- **Fan Community Platform:** Simple forum for content creators to interact with fans

Sports Focus:

- **Sports League Manager:** Track teams, players, schedules, standings

- **Fantasy Sports Tracker:** Manage roster, calculate scores, track matchups
- **Sports News Aggregator:** Collect news from multiple sources, filter by team/sport
- **Workout/Training Log:** Track exercises, sets, reps, progress over time

Real-World Value:

Content creators need tools to manage their presence. Sports fans want community. Both have subscription/premium potential.

Java Example (Content Creator):

```
// Core OOP: Collections (ArrayList, HashMap) + Stream Processing
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.HashMap;

public class ContentCreator {
    private String creatorName;
    private ArrayList<Video> uploadedVideos;
    private HashMap<String, Integer> playlistViews; // Playlist name ->
view count

    public ContentCreator(String creatorName) {
        this.creatorName = creatorName;
        this.uploadedVideos = new ArrayList<>();
        this.playlistViews = new HashMap<>();
    }

    public void uploadVideo(Video video) {
        uploadedVideos.add(video);
        System.out.println("✓ Video uploaded: " + video.getTitle());
    }

    public void createPlaylist(String playlistName, ArrayList<Video>
videos) {
        playlistViews.put(playlistName, 0);
        System.out.println("✓ Playlist created: " + playlistName + " with
" + videos.size() + " videos");
    }

    public ArrayList<Video> getVideosByCategory(String category) {
        ArrayList<Video> filtered = new ArrayList<>();
        for (Video v : uploadedVideos) {
            if (v.getCategory().equals(category)) {
                filtered.add(v);
            }
        }
        return filtered;
    }

    public double getAverageEngagementRate() {
        if (uploadedVideos.isEmpty()) return 0;
    }
}
```

```
        double totalEngagement = 0;
        for (Video v : uploadedVideos) {
            totalEngagement += v.getEngagementRate();
        }
        return totalEngagement / uploadedVideos.size();
    }

    public ArrayList<Video> getTrendingVideos() {
        ArrayList<Video> trending = new ArrayList<>();
        for (Video v : uploadedVideos) {
            if (v.getEngagementRate() > 5.0) { // 5%+ engagement threshold
                trending.add(v);
            }
        }
        return trending;
    }
}

public class Video {
    private String title;
    private String category;
    private int views;
    private int likes;
    private int comments;
    private LocalDateTime uploadDate;
    private int lengthSeconds;

    public Video(String title, String category, int lengthSeconds) {
        this.title = title;
        this.category = category;
        this.lengthSeconds = lengthSeconds;
        this.views = 0;
        this.likes = 0;
        this.comments = 0;
        this.uploadDate = LocalDateTime.now();
    }

    public void recordView() { views++; }
    public void addLike() { likes++; }
    public void addComment() { comments++; }

    public double getEngagementRate() {
        if (views == 0) return 0;
        return ((double)(likes + comments) / views) * 100;
    }

    // Getters
    public String getTitle() { return title; }
    public String getCategory() { return category; }
    public int getViews() { return views; }
    public int getLikes() { return likes; }
    public LocalDateTime getUploadDate() { return uploadDate; }
}
```

Why This Works: Demonstrates ArrayList, HashMap, and practical business logic for growing industries.

Example Demo Usage & Output (Content Creator):

```
public class ContentCreatorDemo {
    public static void main(String[] args) {
        ContentCreator creator = new ContentCreator("TechTalk Sarah");

        System.out.println("=== CONTENT CREATOR DASHBOARD ===\n");

        // Create videos
        Video v1 = new Video("Java Basics Tutorial", "education", 600);
        Video v2 = new Video("10 Coding Tips", "education", 480);
        Video v3 = new Video("My Dev Setup Tour", "vlog", 900);

        creator.uploadVideo(v1);
        creator.uploadVideo(v2);
        creator.uploadVideo(v3);

        // Simulate views and engagement
        System.out.println("\n=== SIMULATING VIEWER INTERACTION ===");
        for (int i = 0; i < 150; i++) v1.recordView();
        for (int i = 0; i < 30; i++) v1.addLike();
        for (int i = 0; i < 8; i++) v1.addComment();

        for (int i = 0; i < 200; i++) v2.recordView();
        for (int i = 0; i < 50; i++) v2.addLike();
        for (int i = 0; i < 12; i++) v2.addComment();

        for (int i = 0; i < 85; i++) v3.recordView();
        for (int i = 0; i < 12; i++) v3.addLike();

        System.out.println("\n=== CHANNEL ANALYTICS ===");
        System.out.println("Average Engagement Rate: " +
            String.format("%.2f", creator.getAverageEngagementRate()) + "%");

        System.out.println("\n=== TRENDING VIDEOS (>5% engagement) ===");
        ArrayList<Video> trending = creator.getTrendingVideos();
        for (Video v : trending) {
            System.out.println("✓ " + v.getTitle() + " - " + v.getViews()
+ " views, " +
                                String.format("%.1f", v.getEngagementRate())
+ "% engagement");
        }
    }
}
```

Output:

```
=== CONTENT CREATOR DASHBOARD ===

✓ Video uploaded: Java Basics Tutorial
✓ Video uploaded: 10 Coding Tips
✓ Video uploaded: My Dev Setup Tour

=== SIMULATING VIEWER INTERACTION ===

=== CHANNEL ANALYTICS ===
Average Engagement Rate: 22.91%

=== TRENDING VIDEOS (>5% engagement) ===
✓ 10 Coding Tips - 200 views, 30.0% engagement
✓ Java Basics Tutorial - 150 views, 25.3% engagement
```

Java Example (Sports):

```
// Core OOP: Polymorphism + Collections
public class SportsTeam {
    private String teamName;
    private String sport;
    private ArrayList<Player> roster;
    private int wins;
    private int losses;

    public SportsTeam(String teamName, String sport) {
        this.teamName = teamName;
        this.sport = sport;
        this.roster = new ArrayList<>();
        this.wins = 0;
        this.losses = 0;
    }

    public void addPlayer(Player player) {
        roster.add(player);
        System.out.println("✓ " + player.getName() + " added to " +
teamName);
    }

    public void recordWin() {
        wins++;
        System.out.println("🏆 " + teamName + " wins! Record: " + wins +
"-" + losses);
    }

    public void recordLoss() {
        losses++;
        System.out.println("😞 " + teamName + " loses. Record: " + wins +
"-" + losses);
    }
}
```



```

    public double getWinPercentage() {
        int totalGames = wins + losses;
        return totalGames == 0 ? 0 : (double)wins / totalGames * 100;
    }

    public Player getTopPerformer() {
        if (roster.isEmpty()) return null;
        Player topPlayer = roster.get(0);
        for (Player p : roster) {
            if (p.getStatScore() > topPlayer.getStatScore()) {
                topPlayer = p;
            }
        }
        return topPlayer;
    }

    public ArrayList<Player> getRoster() { return roster; }
    public int getWins() { return wins; }
    public int getLosses() { return losses; }
}

public class Player {
    private String name;
    private int number;
    private String position;
    private int statScore; // Points/runs/goals depending on sport

    public Player(String name, int number, String position) {
        this.name = name;
        this.number = number;
        this.position = position;
        this.statScore = 0;
    }

    public void addStatScore(int points) {
        this.statScore += points;
    }

    // Getters
    public String getName() { return name; }
    public int getNumber() { return number; }
    public String getPosition() { return position; }
    public int getStatScore() { return statScore; }
}

```

Why This Works: Demonstrates ArrayList, HashMap, and practical business logic for growing industries.

Example Demo Usage & Output (Sports):

```
public class SportsDemo {
    public static void main(String[] args) {
        SportsTeam lakers = new SportsTeam("Lakers", "Basketball");

        System.out.println("=== BASKETBALL SEASON ===\n");

        // Add players
        Player lebron = new Player("LeBron James", 23, "Forward");
        Player ad = new Player("Anthony Davis", 3, "Center");
        Player rui = new Player("Rui Hachimura", 28, "Forward");

        lakers.addPlayer(lebron);
        lakers.addPlayer(ad);
        lakers.addPlayer(rui);

        System.out.println("\n=== GAME RESULTS ===");

        // Game 1
        System.out.println("Game 1: Lakers vs Celtics");
        lebron.addStatScore(28);
        ad.addStatScore(22);
        rui.addStatScore(15);
        lakers.recordWin();

        // Game 2
        System.out.println("Game 2: Lakers vs Warriors");
        lebron.addStatScore(31);
        ad.addStatScore(18);
        rui.addStatScore(12);
        lakers.recordWin();

        // Game 3
        System.out.println("Game 3: Lakers vs Suns");
        lebron.addStatScore(25);
        ad.addStatScore(20);
        lakers.recordLoss();

        System.out.println("\n=== SEASON STATS ===");
        System.out.println("Record: " + lakers.getWins() + "-" +
lakers.getLosses());
        System.out.println("Win %: " + String.format("%.1f",
lakers.getWinPercentage()) + "%");

        Player topPlayer = lakers.getTopPerformer();
        System.out.println("\nTop Performer: " + topPlayer.getName() + " -
" + topPlayer.getStatScore() + " total points");
    }
}
```

Output:

```
=== BASKETBALL SEASON ===
```

- ✓ LeBron James added to Lakers
- ✓ Anthony Davis added to Lakers
- ✓ Rui Hachimura added to Lakers

```
=== GAME RESULTS ===
```

Game 1: Lakers vs Celtics

🎉 Lakers wins! Record: 1-0

Game 2: Lakers vs Warriors

🎉 Lakers wins! Record: 2-0

Game 3: Lakers vs Suns

😞 Lakers loses. Record: 2-1

```
=== SEASON STATS ===
```

Record: 2-1

Win %: 66.7%

Top Performer: LeBron James - 84 total points

6. FASHION/FASHION-TECH APPLICATION

Project Ideas:

- **Virtual Wardrobe Organizer:** Catalog clothes, create outfits, track wear frequency
- **Size Conversion Tool:** Quick converter for different international sizing standards
- **Outfit Recommender:** Suggest outfits based on occasion and weather
- **Fashion Brand Inventory:** Track apparel by color, size, price point

Real-World Value:

Fashion is a \$1.5 trillion industry. Tools that help consumers save time organizing their wardrobe or brands manage inventory are valuable.

Java Example:

```
// Core OOP: Encapsulation + Collections + Sorting
import java.util.ArrayList;
import java.util.Comparator;

public class VirtualWardrobe {
    private ArrayList<Clothing> items;
    private String ownerName;

    public VirtualWardrobe(String ownerName) {
        this.ownerName = ownerName;
        this.items = new ArrayList<>();
    }
}
```

```
public void addClothing(Clothing item) {
    items.add(item);
    System.out.println("✓ Added: " + item.getDescription());
}

public ArrayList<Clothing> getItemsByCategory(String category) {
    ArrayList<Clothing> filtered = new ArrayList<>();
    for (Clothing item : items) {
        if (item.getCategory().equals(category)) {
            filtered.add(item);
        }
    }
    return filtered;
}

public ArrayList<Clothing> getMostWornItems() {
    ArrayList<Clothing> sorted = new ArrayList<>(items);
    sorted.sort(Comparator.comparingInt(Clothing::getWearCount).reversed());
    return sorted;
}

public ArrayList<Outfit> suggestOutfits(String occasion) {
    ArrayList<Outfit> outfits = new ArrayList<>();

    // Simple logic: match appropriate items by occasion
    ArrayList<Clothing> tops = getItemsByCategory("top");
    ArrayList<Clothing> bottoms = getItemsByCategory("bottom");

    for (Clothing top : tops) {
        if (top.getOccasionRating(occasion) >= 7) {
            for (Clothing bottom : bottoms) {
                if (bottom.getOccasionRating(occasion) >= 7) {
                    outfits.add(new Outfit(top, bottom, occasion));
                }
            }
        }
    }
    return outfits;
}

public int getTotalValue() {
    int total = 0;
    for (Clothing item : items) {
        total += item.getPrice();
    }
    return total;
}

}

public class Clothing {
    private String name;
    private String category; // "top", "bottom", "shoes", "accessory"
```

```
private String color;
private String size;
private double price;
private int wearCount;
private HashMap<String, Integer> occasionRatings; // occasion ->
rating (1-10)

public Clothing(String name, String category, String color, String
size, double price) {
    this.name = name;
    this.category = category;
    this.color = color;
    this.size = size;
    this.price = price;
    this.wearCount = 0;
    this.occasionRatings = new HashMap<>();
}

public void recordWear() {
    wearCount++;
}

public void setOccasionRating(String occasion, int rating) {
    occasionRatings.put(occasion, rating);
}

public int getOccasionRating(String occasion) {
    return occasionRatings.getOrDefault(occasion, 5); // default
neutral rating
}

public String getDescription() {
    return color + " " + name + " (" + size + ") - $" + price;
}

// Getters
public String getCategory() { return category; }
public String getColor() { return color; }
public int getPrice() { return (int)price; }
public int getWearCount() { return wearCount; }
}

public class Outfit {
    private Clothing top;
    private Clothing bottom;
    private String occasion;

    public Outfit(Clothing top, Clothing bottom, String occasion) {
        this.top = top;
        this.bottom = bottom;
        this.occasion = occasion;
    }

    public void displayOutfit() {
```

```

        System.out.println("👗 Outfit for " + occasion + ":");
        System.out.println("  Top: " + top.getDescription());
        System.out.println("  Bottom: " + bottom.getDescription());
    }
}

```

Why This Works: Uses ArrayList, HashMap for sorting—practical for the fashion industry.

Demo Usage & Output:

```

public class FashionDemo {
    public static void main(String[] args) {
        VirtualWardrobe wardrobe = new VirtualWardrobe("Maria");

        System.out.println("=== VIRTUAL WARDROBE ===\n");

        // Add clothing items
        Clothing blueBlouse = new Clothing("Silk Blouse", "top", "Blue",
            "M", 45.99);
        Clothing whiteShirt = new Clothing("Cotton Shirt", "top", "White",
            "M", 25.99);
        Clothing blackPants = new Clothing("Dress Pants", "bottom",
            "Black", "M", 65.00);
        Clothing jeans = new Clothing("Jeans", "bottom", "Blue", "M",
            55.00);

        wardrobe.addClothing(blueBlouse);
        wardrobe.addClothing(whiteShirt);
        wardrobe.addClothing(blackPants);
        wardrobe.addClothing(jeans);

        // Set occasion ratings
        blueBlouse.setOccasionRating("work", 9);
        blueBlouse.setOccasionRating("casual", 6);
        whiteShirt.setOccasionRating("casual", 9);
        whiteShirt.setOccasionRating("work", 7);
        blackPants.setOccasionRating("work", 10);
        blackPants.setOccasionRating("casual", 5);
        jeans.setOccasionRating("casual", 10);
        jeans.setOccasionRating("work", 3);

        // Record wearing items
        System.out.println("\n=== TRACKING USAGE ===");
        for (int i = 0; i < 5; i++) blueBlouse.recordWear();
        for (int i = 0; i < 8; i++) jeans.recordWear();
        for (int i = 0; i < 2; i++) blackPants.recordWear();
        System.out.println("Tracked wear history\n");

        // Get outfit suggestions
        System.out.println("=== OUTFIT SUGGESTIONS FOR WORK ===");
        ArrayList<Outfit> workOutfits = wardrobe.suggestOutfits("work");
    }
}

```

```

        for (Outfit o : workOutfits) {
            o.displayOutfit();
        }

        System.out.println("\n=== WARDROBE STATS ===");
        System.out.println("Total Wardrobe Value: $" +
wardrobe.getTotalValue());
        System.out.println("\nMost Worn Items:");
        ArrayList<Clothing> worn = wardrobe.getMostWornItems();
        for (Clothing c : worn) {
            if (c.getWearCount() > 0) {
                System.out.println("- " + c.getDescription() + " (worn " +
c.getWearCount() + " times)");
            }
        }
    }
}

```

Output:

```

=== VIRTUAL WARDROBE ===

✓ Added: Blue Silk Blouse (M) - $45.99
✓ Added: White Cotton Shirt (M) - $25.99
✓ Added: Black Dress Pants (M) - $65.0
✓ Added: Blue Jeans (M) - $55.0

=== TRACKING USAGE ===
Tracked wear history

=== OUTFIT SUGGESTIONS FOR WORK ===
👔 Outfit for work:
  Top: Blue Silk Blouse (M) - $45.99
  Bottom: Black Dress Pants (M) - $65.0

=== WARDROBE STATS ===
Total Wardrobe Value: $191
Most Worn Items:
- Blue Jeans (M) - $55.0 (worn 8 times)
- Blue Silk Blouse (M) - $45.99 (worn 5 times)

```

7. FINANCIAL TECHNOLOGY (FINTECH)

Project Ideas:

- **Expense Tracker with Budget Goals:** Log expenses, categorize, set alerts
- **Personal Finance Dashboard:** Income vs. expenses, savings tracking, monthly reports
- **Bill Reminder System:** Track recurring bills, send payment reminders
- **Investment Portfolio Tracker:** Simulate stock purchases, track gains/losses

Real-World Value:

Personal finance is a pain point. Fintech tools are highly monetizable through subscriptions or premium features.

Java Example:

```
// Core OOP: Collections (ArrayList, HashMap) + Data Aggregation
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.HashMap;

public class BudgetTracker {
    private String userEmail;
    private HashMap<String, Double> budgets; // category -> budget limit
    private ArrayList<Transaction> transactions;
    private double income;

    public BudgetTracker(String userEmail, double monthlyIncome) {
        this.userEmail = userEmail;
        this.income = monthlyIncome;
        this.budgets = new HashMap<>();
        this.transactions = new ArrayList<>();
    }

    public void setBudgetLimit(String category, double limit) {
        budgets.put(category, limit);
        System.out.println("✓ Budget set for " + category + ": $" +
limit);
    }

    public void addExpense(String category, double amount, String
description) {
        transactions.add(new Transaction(category, amount, description,
LocalDate.now()));

        // Check if budget exceeded
        double spent = getSpentByCategory(category);
        if (budgets.containsKey(category)) {
            double limit = budgets.get(category);
            if (spent > limit) {
                System.out.println("⚠ ALERT: " + category + " budget
exceeded! $" +
                                String.format("%.2f", spent) + " / $" +
limit);
            }
        }
    }

    public double getSpentByCategory(String category) {
        double spent = 0;
        for (Transaction t : transactions) {
```



```
        if (t.getCategory().equals(category)) {
            spent += t.getAmount();
        }
    }
    return spent;
}

public double getTotalSpent() {
    double total = 0;
    for (Transaction t : transactions) {
        total += t.getAmount();
    }
    return total;
}

public double getSavings() {
    return income - getTotalSpent();
}

public void displayMonthlyReport() {
    System.out.println("\n📊 MONTHLY FINANCIAL REPORT");
    System.out.println("Income: $" + String.format("%.2f", income));
    System.out.println("Total Expenses: $" + String.format("%.2f",
getTotalSpent()));
    System.out.println("Remaining: $" + String.format("%.2f",
getSavings()));
    System.out.println("\nBy Category:");
    for (String category : budgets.keySet()) {
        double spent = getSpentByCategory(category);
        double limit = budgets.get(category);
        System.out.println("  " + category + ": $" +
String.format("%.2f", spent) +
        " / $" + limit);
    }
}

}

public class Transaction {
    private String category;
    private double amount;
    private String description;
    private LocalDate date;

    public Transaction(String category, double amount, String description,
LocalDate date) {
        this.category = category;
        this.amount = amount;
        this.description = description;
        this.date = date;
    }

    public String getCategory() { return category; }
    public double getAmount() { return amount; }
    public String getDescription() { return description; }
```

```
    public LocalDate getDate() { return date; }  
}
```

Why This Works: Demonstrates HashMap for category tracking, ArrayList for transaction history, and practical personal finance logic.

Example Demo Usage & Output:

```
public class BudgetTrackerDemo {  
    public static void main(String[] args) {  
        BudgetTracker tracker = new BudgetTracker("alex@email.com",  
3500.00);  
  
        System.out.println("=== PERSONAL BUDGET TRACKER ===\n");  
  
        // Set budgets  
        tracker.setBudgetLimit("Groceries", 400.00);  
        tracker.setBudgetLimit("Dining", 150.00);  
        tracker.setBudgetLimit("Entertainment", 200.00);  
        tracker.setBudgetLimit("Utilities", 300.00);  
  
        System.out.println("\n=== LOGGING EXPENSES ===");  
        tracker.addExpense("Groceries", 85.50, "Whole Foods - weekly  
shopping");  
        tracker.addExpense("Groceries", 62.30, "Trader Joe's");  
        tracker.addExpense("Dining", 45.00, "Lunch with Sarah");  
        tracker.addExpense("Dining", 88.00, "Dinner at Italian  
restaurant");  
        tracker.addExpense("Entertainment", 15.00, "Movie ticket");  
        tracker.addExpense("Utilities", 125.00, "Electric bill");  
        tracker.addExpense("Utilities", 65.00, "Internet bill");  
        tracker.addExpense("Dining", 35.00, "Breakfast");  
  
        System.out.println("\n=== CATEGORY BREAKDOWN ===");  
        System.out.println("Groceries spent: $" + String.format("%.2f",  
tracker.getSpentByCategory("Groceries")));  
        System.out.println("Dining spent: $" + String.format("%.2f",  
tracker.getSpentByCategory("Dining")));  
        System.out.println("Entertainment spent: $" +  
String.format("%.2f", tracker.getSpentByCategory("Entertainment")));  
  
        // Display full report  
        tracker.displayMonthlyReport();  
    }  
}
```

Output:

```
=== PERSONAL BUDGET TRACKER ===

✓ Budget set for Groceries: $400.0
✓ Budget set for Dining: $150.0
✓ Budget set for Entertainment: $200.0
✓ Budget set for Utilities: $300.0

=== LOGGING EXPENSES ===
⚠ ALERT: Dining budget exceeded! $168.00 / $150.0

=== CATEGORY BREAKDOWN ===
Groceries spent: $147.80
Dining spent: $168.00
Entertainment spent: $15.00

🇮🇹 MONTHLY FINANCIAL REPORT
Income: $3500.00
Total Expenses: $520.80
Remaining: $2979.20

By Category:
  Groceries: $147.80 / $400.00
  Dining: $168.00 / $150.00
  Entertainment: $15.00 / $200.00
  Utilities: $190.00 / $300.00
```

8. EDUCATION/E-LEARNING

Project Ideas:

- **Quiz Generator & Scorer:** Create quizzes, auto-score answers, track results
- **Flashcard System:** Create decks, randomize questions, track progress
- **Study Session Timer:** Pomodoro-style study tracker with break reminders
- **Course Progress Dashboard:** Track modules completed, assignments submitted, grades

Real-World Value:

EdTech is a massive market. Platforms that help students study are always in demand, especially subscription-based or with premium features.

Java Example:

```
// Core OOP: Inheritance + Polymorphism + Collections
import java.util.ArrayList;
import java.util.Collections;

public class Quiz {
    private String title;
    private ArrayList<Question> questions;
```

```
private ArrayList<QuizResult> results;

public Quiz(String title) {
    this.title = title;
    this.questions = new ArrayList<>();
    this.results = new ArrayList<>();
}

public void addQuestion(Question q) {
    questions.add(q);
}

public QuizResult administeQuiz(Student student, ArrayList<String>
studentAnswers) {
    int score = 0;
    ArrayList<String> feedback = new ArrayList<>();

    for (int i = 0; i < questions.size(); i++) {
        if (i < studentAnswers.size()) {
            String answer = studentAnswers.get(i);
            Question q = questions.get(i);
            if (q.isCorrect(answer)) {
                score++;
                feedback.add("✓ Q" + (i+1) + " Correct!");
            } else {
                feedback.add("✗ Q" + (i+1) + " Incorrect. Correct
answer: " + q.getCorrectAnswer());
            }
        }
    }

    double percentage = (double)score / questions.size() * 100;
    QuizResult result = new QuizResult(student, percentage, feedback);
    results.add(result);

    return result;
}

public double getAverageScore() {
    if (results.isEmpty()) return 0;
    double total = 0;
    for (QuizResult r : results) {
        total += r.getPercentage();
    }
    return total / results.size();
}

public abstract class Question {
    protected String questionText;
    protected String correctAnswer;

    public Question(String questionText, String correctAnswer) {
        this.questionText = questionText;
    }
}
```

```
        this.correctAnswer = correctAnswer;
    }

    public abstract boolean isCorrect(String answer);
    public String getCorrectAnswer() { return correctAnswer; }
    public String getQuestionText() { return questionText; }
}

public class MultipleChoice extends Question {
    private ArrayList<String> options;

    public MultipleChoice(String questionText, ArrayList<String> options,
String correctAnswer) {
        super(questionText, correctAnswer);
        this.options = options;
    }

    @Override
    public boolean isCorrect(String answer) {
        return answer.equalsIgnoreCase(correctAnswer);
    }

    public ArrayList<String> getOptions() { return options; }
}

public class ShortAnswer extends Question {
    public ShortAnswer(String questionText, String correctAnswer) {
        super(questionText, correctAnswer);
    }

    @Override
    public boolean isCorrect(String answer) {
        // Simple fuzzy matching (strip whitespace, case-insensitive)
        return answer.trim().equalsIgnoreCase(correctAnswer.trim());
    }
}

public class QuizResult {
    private Student student;
    private double percentage;
    private ArrayList<String> feedback;

    public QuizResult(Student student, double percentage,
ArrayList<String> feedback) {
        this.student = student;
        this.percentage = percentage;
        this.feedback = feedback;
    }

    public void displayResult() {
        System.out.println("\n📊 Quiz Results for " + student.getName());
        System.out.println("Score: " + String.format("%.1f", percentage) +
"%");
        for (String f : feedback) {
```

```

        System.out.println(f);
    }
}

public double getPercentage() { return percentage; }
}

public class Student {
    private String name;
    private String studentId;

    public Student(String name, String studentId) {
        this.name = name;
        this.studentId = studentId;
    }

    public String getName() { return name; }
    public String getStudentId() { return studentId; }
}

```

Why This Works: Demonstrates inheritance/polymorphism with Question types, ArrayList for quiz management, practical education use case.

Example Demo Usage & Output:

```

public class QuizDemo {
    public static void main(String[] args) {
        Quiz javaQuiz = new Quiz("Java Basics");

        System.out.println("=== CREATING QUIZ ===\n");

        // Create questions
        ArrayList<String> options1 = new ArrayList<>();
        options1.add("A) Stores multiple values");
        options1.add("B) Stores one value only");
        options1.add("C) Not used in Java");

        javaQuiz.addQuestion(new MultipleChoice(
            "What is an ArrayList?", options1, "A) Stores multiple
values"));

        javaQuiz.addQuestion(new ShortAnswer(
            "What keyword creates a new ArrayList?", "new"));

        javaQuiz.addQuestion(new ShortAnswer(
            "What method adds an item to an ArrayList?", "add"));

        System.out.println("Created 3-question quiz\n");

        // Take quiz
        System.out.println("=== STUDENT TAKING QUIZ ===");
    }
}

```

```

Student student1 = new Student("Emma Johnson", "S001");
ArrayList<String> answers = new ArrayList<>();
answers.add("A) Stores multiple values");
answers.add("new");
answers.add("add");

QuizResult result = javaQuiz.administeQuiz(student1, answers);
result.displayResult();

// Another student
System.out.println("\n=== SECOND STUDENT ===");
Student student2 = new Student("Marcus Davis", "S002");
ArrayList<String> answers2 = new ArrayList<>();
answers2.add("B) Stores one value only"); // Wrong
answers2.add("new"); // Correct
answers2.add("add"); // Correct

QuizResult result2 = javaQuiz.administeQuiz(student2, answers2);
result2.displayResult();

// Class average
System.out.println("\n=== CLASS STATISTICS ===");
System.out.println("Average Score: " + String.format("%.1f",
javaQuiz.getAverageScore()) + "%");
}
}

```

Output:

```

=== CREATING QUIZ ===

Created 3-question quiz

=== STUDENT TAKING QUIZ ===
✓ Q1 Correct!
✓ Q2 Correct!
✓ Q3 Correct!

📝 Quiz Results for Emma Johnson
Score: 100.0%
✓ Q1 Correct!
✓ Q2 Correct!
✓ Q3 Correct!

=== SECOND STUDENT ===
x Q1 Incorrect. Correct answer: A) Stores multiple values
✓ Q2 Correct!
✓ Q3 Correct!

📝 Quiz Results for Marcus Davis
Score: 66.7%
x Q1 Incorrect. Correct answer: A) Stores multiple values

```

```
✓ Q2 Correct!
✓ Q3 Correct!

=== CLASS STATISTICS ===
Average Score: 83.3%
```

9. SMART HOME/IOT (SIMULATED)

Project Ideas:

- **Smart Home Control Panel:** Control lights, temperature, security, locks
- **Energy Usage Monitor:** Track power consumption, estimate costs
- **Smart Lighting System:** Schedule lights, automate brightness by time of day
- **Home Security Simulator:** Arm/disarm system, log sensor alerts, track history

Real-World Value:

IoT is one of the fastest-growing tech sectors. Smart home simulation is scalable to actual hardware.

Java Example:

```
// Core OOP: Polymorphism + State Management + Collections
public abstract class SmartDevice {
    protected String deviceName;
    protected boolean isOn;
    protected String location;

    public SmartDevice(String deviceName, String location) {
        this.deviceName = deviceName;
        this.location = location;
        this.isOn = false;
    }

    public abstract void turnOn();
    public abstract void turnOff();
    public abstract void displayStatus();

    public String getDeviceName() { return deviceName; }
    public boolean isOn() { return isOn; }
}

public class SmartLight extends SmartDevice {
    private int brightness; // 0-100

    public SmartLight(String deviceName, String location) {
        super(deviceName, location);
        this.brightness = 0;
    }

    @Override
```



```
    public void turnOn() {
        isOn = true;
        brightness = 100;
        System.out.println("💡 " + deviceName + " turned ON (100%
brightness)");
    }

    @Override
    public void turnOff() {
        isOn = false;
        brightness = 0;
        System.out.println("💡 " + deviceName + " turned OFF");
    }

    public void setBrightness(int level) {
        if (level >= 0 && level <= 100) {
            brightness = level;
            isOn = (level > 0);
            System.out.println("💡 " + deviceName + " brightness set to "
+ level + "%");
        }
    }

    @Override
    public void displayStatus() {
        System.out.println("[ " + location + " ] " + deviceName + ": " +
            (isOn ? "ON (" + brightness + "%)" : "OFF"));
    }
}

public class SmartThermostat extends SmartDevice {
    private int currentTemp;
    private int targetTemp;

    public SmartThermostat(String deviceName, String location, int
initialTemp) {
        super(deviceName, location);
        this.currentTemp = initialTemp;
        this.targetTemp = initialTemp;
    }

    @Override
    public void turnOn() {
        isOn = true;
        System.out.println("🔥 " + deviceName + " turned ON");
    }

    @Override
    public void turnOff() {
        isOn = false;
        System.out.println("🔥 " + deviceName + " turned OFF");
    }

    public void setTargetTemp(int target) {
```

```

        targetTemp = target;
        System.out.println("🔥 " + deviceName + " target set to " + target
+ "°F");
    }

    @Override
    public void displayStatus() {
        System.out.println("[ " + location + " ] " + deviceName + ": " +
            currentTemp + "°F (Target: " + targetTemp +
"°F)");
    }
}

public class SmartHome {
    private ArrayList<SmartDevice> devices;
    private String homeAddress;

    public SmartHome(String homeAddress) {
        this.homeAddress = homeAddress;
        this.devices = new ArrayList<>();
    }

    public void addDevice(SmartDevice device) {
        devices.add(device);
        System.out.println("✓ " + device.getDeviceName() + " added to
home");
    }

    public void turnOnAll() {
        for (SmartDevice device : devices) {
            device.turnOn();
        }
    }

    public void turnOffAll() {
        for (SmartDevice device : devices) {
            device.turnOff();
        }
    }

    public void displayAllStatus() {
        System.out.println("\n🏠 Smart Home Status: " + homeAddress);
        System.out.println("=====");
        for (SmartDevice device : devices) {
            device.displayStatus();
        }
        System.out.println("=====");
    }

    public int getActiveDeviceCount() {
        int count = 0;
        for (SmartDevice device : devices) {
            if (device.isOn()) count++;
        }
    }
}

```

```
        return count;
    }
}
```

Why This Works: Shows polymorphism with different device types, ArrayList for device management, scalable architecture.

Example Demo Usage & Output:

```
public class SmartHomeDemo {
    public static void main(String[] args) {
        SmartHome myHome = new SmartHome("123 Main Street");

        System.out.println("=== SETTING UP SMART HOME ===\n");

        // Add devices
        SmartLight livingRoomLight = new SmartLight("Living Room Light",
"Living Room");
        SmartLight bedroomLight = new SmartLight("Bedroom Light",
"Bedroom");
        SmartThermostat thermostat = new SmartThermostat("Main
Thermostat", "Hallway", 72);

        myHome.addDevice(livingRoomLight);
        myHome.addDevice(bedroomLight);
        myHome.addDevice(thermostat);

        System.out.println("\n=== CONTROLLING DEVICES ===");
        livingRoomLight.turnOn();
        livingRoomLight.setBrightness(80);
        bedroomLight.turnOn();
        bedroomLight.setBrightness(30);
        thermostat.turnOn();
        thermostat.setTargetTemp(70);

        System.out.println("\n=== HOME STATUS ===");
        myHome.displayAllStatus();
        System.out.println("Active devices: " +
myHome.getActiveDeviceCount());

        System.out.println("\n=== EVENING MODE ===");
        System.out.println("Dimming lights and adjusting thermostat...");
        livingRoomLight.setBrightness(40);
        bedroomLight.setBrightness(10);
        thermostat.setTargetTemp(68);

        System.out.println("\n=== GOOD NIGHT - TURNING OFF ALL ===");
        myHome.turnOffAll();
        myHome.displayAllStatus();
        System.out.println("Active devices: " +
myHome.getActiveDeviceCount());
    }
}
```

```
}  
}
```

Output:

```
=== SETTING UP SMART HOME ===  
  
✓ Living Room Light added to home  
✓ Bedroom Light added to home  
✓ Main Thermostat added to home  
  
=== CONTROLLING DEVICES ===  
💡 Living Room Light turned ON (100% brightness)  
💡 Living Room Light brightness set to 80%  
💡 Bedroom Light turned ON (100% brightness)  
💡 Bedroom Light brightness set to 30%  
🌡 Main Thermostat turned ON  
🌡 Main Thermostat target set to 70°F  
  
=== HOME STATUS ===  
  
📶 Smart Home Status: 123 Main Street  
=====
```

Room	Device	Status
Living Room	Living Room Light	ON (80%)
Bedroom	Bedroom Light	ON (30%)
Hallway	Main Thermostat	72°F (Target: 70°F)

```
=====
```

Active devices: 3

```
=== EVENING MODE ===  
Dimming lights and adjusting thermostat...  
💡 Living Room Light brightness set to 40%  
💡 Bedroom Light brightness set to 10%  
🌡 Main Thermostat target set to 68°F  
  
=== GOOD NIGHT – TURNING OFF ALL ===  
💡 Living Room Light turned OFF  
💡 Bedroom Light turned OFF  
🌡 Main Thermostat turned OFF  
  
📶 Smart Home Status: 123 Main Street  
=====
```

Room	Device	Status
Living Room	Living Room Light	OFF
Bedroom	Bedroom Light	OFF
Hallway	Main Thermostat	72°F (Target: 68°F)

```
=====
```

Active devices: 0

10. TRANSPORTATION/LOGISTICS

Project Ideas:

- **Delivery Route Optimizer:** Plan efficient routes, track packages
- **Fleet Management System:** Track vehicle locations, fuel, maintenance schedules
- **Public Transit Tracker:** Real-time updates on bus/train locations and delays
- **Ride-Sharing Dashboard:** Match riders, calculate fares, track trips

Real-World Value:

Logistics is a \$12+ trillion global industry. Routing and tracking tools save companies millions in fuel and labor.

Java Example:

```
// Core OOP: Collections + Algorithms + State Tracking
import java.util.ArrayList;

public class DeliveryPackage {
    private String trackingId;
    private String address;
    private String status; // "pending", "in_transit", "delivered"
    private double weight;
    private double priority; // 1-10, affects routing

    public DeliveryPackage(String trackingId, String address, double
weight, double priority) {
        this.trackingId = trackingId;
        this.address = address;
        this.weight = weight;
        this.priority = priority;
        this.status = "pending";
    }

    public String getTrackingId() { return trackingId; }
    public String getAddress() { return address; }
    public String getStatus() { return status; }
    public void setStatus(String status) { this.status = status; }
    public double getPriority() { return priority; }
    public double getWeight() { return weight; }
}

public class DeliveryRoute {
    private String routeId;
    private ArrayList<DeliveryPackage> packages;
    private String driverId;
    private double totalDistance;
    private boolean isActive;

    public DeliveryRoute(String routeId, String driverId) {
        this.routeId = routeId;
        this.driverId = driverId;
        this.packages = new ArrayList<>();
    }
}
```

```
        this.totalDistance = 0;
        this.isActive = false;
    }

    public void addPackage(DeliveryPackage pkg) {
        // Prioritize high-priority packages at the front
        packages.add(pkg);
        packages.sort((p1, p2) -> Double.compare(p2.getPriority(),
p1.getPriority()));
        System.out.println("✓ Package " + pkg.getTrackingId() + " added to
route");
    }

    public void startRoute() {
        isActive = true;
        System.out.println("🚚 Route " + routeId + " started with " +
packages.size() + " deliveries");
    }

    public void completeDelivery(String trackingId) {
        for (DeliveryPackage pkg : packages) {
            if (pkg.getTrackingId().equals(trackingId)) {
                pkg.setStatus("delivered");
                System.out.println("✓ Delivery completed: " + trackingId);
                return;
            }
        }
    }

    public double getAverageWeight() {
        if (packages.isEmpty()) return 0;
        double total = 0;
        for (DeliveryPackage pkg : packages) {
            total += pkg.getWeight();
        }
        return total / packages.size();
    }

    public ArrayList<DeliveryPackage> getPendingDeliveries() {
        ArrayList<DeliveryPackage> pending = new ArrayList<>();
        for (DeliveryPackage pkg : packages) {
            if (!pkg.getStatus().equals("delivered")) {
                pending.add(pkg);
            }
        }
        return pending;
    }

    public void displayRoute() {
        System.out.println("\n🚚 Route " + routeId + " - Driver: " +
driverId);
        System.out.println("Status: " + (isActive ? "ACTIVE" :
"INACTIVE"));
        System.out.println("Deliveries (" + packages.size() + "):");
    }
}
```

```

        for (DeliveryPackage pkg : packages) {
            System.out.println("  → " + pkg.getTrackingId() + ": " +
pkg.getAddress() +
                " (" + pkg.getStatus() + ")");
        }
    }
}

public class LogisticsCompany {
    private String companyName;
    private ArrayList<DeliveryRoute> activeRoutes;
    private ArrayList<DeliveryPackage> allPackages;

    public LogisticsCompany(String companyName) {
        this.companyName = companyName;
        this.activeRoutes = new ArrayList<>();
        this.allPackages = new ArrayList<>();
    }

    public void createRoute(DeliveryRoute route) {
        activeRoutes.add(route);
        System.out.println("✓ Route created: " + route.routeId);
    }

    public void trackPackage(String trackingId) {
        for (DeliveryPackage pkg : allPackages) {
            if (pkg.getTrackingId().equals(trackingId)) {
                System.out.println("📦 Tracking " + trackingId + ": " +
pkg.getStatus());
                return;
            }
        }
        System.out.println("Package not found");
    }

    public int getTotalDeliveries() {
        int count = 0;
        for (DeliveryRoute route : activeRoutes) {
            count += route.packages.size();
        }
        return count;
    }
}

```

Why This Works: Demonstrates ArrayList sorting and practical logistics algorithms.

Example Demo Usage & Output:

```

public class LogisticsDemo {
    public static void main(String[] args) {
        LogisticsCompany fedex = new LogisticsCompany("FastFreight

```

```
Delivery");
    DeliveryRoute route1 = new DeliveryRoute("RT-001", "Driver: John
Smith");

    System.out.println("=== DELIVERY ROUTE SETUP ===\n");

    // Create packages with different priorities
    DeliveryPackage pkg1 = new DeliveryPackage("TRK-101", "123 Oak
St", 2.5, 7);
    DeliveryPackage pkg2 = new DeliveryPackage("TRK-102", "456 Maple
Ave", 1.0, 3);
    DeliveryPackage pkg3 = new DeliveryPackage("TRK-103", "789 Pine
Rd", 5.0, 10);
    DeliveryPackage pkg4 = new DeliveryPackage("TRK-104", "321 Elm
St", 0.5, 2);

    // Add packages (automatically sorted by priority)
    System.out.println("Adding packages to route...");
    route1.addPackage(pkg1);
    route1.addPackage(pkg2);
    route1.addPackage(pkg3);
    route1.addPackage(pkg4);

    fedex.createRoute(route1);

    // Display route before starting
    System.out.println();
    route1.displayRoute();

    // Start delivery
    System.out.println("\n=== STARTING DELIVERIES ===");
    route1.startRoute();

    // Complete deliveries
    System.out.println("\nCompleting deliveries in order...");
    route1.completeDelivery("TRK-103");
    route1.completeDelivery("TRK-101");
    route1.completeDelivery("TRK-102");

    // Show pending deliveries
    System.out.println("\n=== REMAINING DELIVERIES ===");
    ArrayList<DeliveryPackage> pending =
route1.getPendingDeliveries();
    System.out.println("Pending: " + pending.size());
    for (DeliveryPackage p : pending) {
        System.out.println("  → " + p.getTrackingId() + " at " +
p.getAddress());
    }

    System.out.println("\n=== ROUTE STATS ===");
    System.out.println("Average package weight: " +
String.format("%.2f", route1.getAverageWeight()) + " lbs");
    System.out.println("Total packages: " + route1.packages.size());
```



```
}  
}
```

Output:

=== DELIVERY ROUTE SETUP ===

Adding packages to route...

- ✓ Package TRK-101 added to route
- ✓ Package TRK-102 added to route
- ✓ Package TRK-103 added to route
- ✓ Package TRK-104 added to route
- ✓ Route created: RT-001

 Route RT-001 – Driver: Driver: John Smith
Status: INACTIVE

Deliveries (4):

- TRK-103: 789 Pine Rd (pending)
- TRK-101: 123 Oak St (pending)
- TRK-102: 456 Maple Ave (pending)
- TRK-104: 321 Elm St (pending)

=== STARTING DELIVERIES ===

 Route RT-001 started with 4 deliveries

Completing deliveries in order...

- ✓ Delivery completed: TRK-103
- ✓ Delivery completed: TRK-101
- ✓ Delivery completed: TRK-102

=== REMAINING DELIVERIES ===

Pending: 1

- TRK-104 at 321 Elm St

=== ROUTE STATS ===

Average package weight: 2.25 lbs

Total packages: 4